Experiment 9

Name: P. Hithaishi **Course Code: MLA0305**

Regno: 192425196 **Slot: B**

Deep Q-Network (DQN) Implementation using TensorFlow and Keras

AIM

To implement a Deep Q-Network (DQN) using TensorFlow and Keras for solving a Reinforcement Learning problem.

PROCEDURE

1. Set up the TensorFlow, Keras and Gymnasium environment.
2. Import the required libraries and modules.
3. Create a suitable Gymnasium Reinforcement Learning environment.
4. Define the state space and action space.
5. Construct the neural network for estimating Q-values.
6. Initialize the DQN model and its parameters.
7. Allow the agent to interact with the environment and collect experiences.
8. Train the neural network using the collected experiences.
9. Repeat the training process for multiple episodes.
10. Evaluate the trained DQN agent and observe its learning performance.

CODE

```
!pip -q install gymnasium

# Import libraries import gymnasium
as gym import numpy as np import
tensorflow as tf from tensorflow
import keras from tensorflow.keras
import layers import random from
collections import deque import
matplotlib.pyplot as plt

env = gym.make("CartPole-v1")

state_size = int(env.observation_space.shape[0]) action_size
= int(env.action_space.n)
```

```
print("State Size :", state_size) print("Action
Size:", action_size)

def create_model():

    model = keras.Sequential([
layers.Input(shape=(state_size,)),         layers.Dense(64,
activation="relu"),         layers.Dense(64,
activation="relu"),         layers.Dense(action_size,
activation="linear")
    ])

    model.compile(
optimizer=keras.optimizers.Adam(learning_rate=0.001),
loss="mse"
    )

    return model


# Main DQN network
model = create_model()

# Target network target_model
= create_model()

# Copy weights
target_model.set_weights(model.get_weights())

print("\nDQN model created successfully!")

# ------------------------------------------------
# 3. REPLAY MEMORY
# ------------------------------------------------

memory = deque(maxlen=20000)

# ------------------------------------------------
# 4. PARAMETERS
# ------------------------------------------------

gamma = 0.95
```

```
epsilon     =     1.0
epsilon_min   =   0.01
epsilon_decay = 0.995

batch_size = 32 episodes
= 100 rewards_history =
[]

# ------------------------------------------------
# 5. TRAIN DQN
# ------------------------------------------------

for episode in range(episodes):

    state, info = env.reset()

    state = np.asarray(state, dtype=np.float32)

    total_reward = 0
done = False

    while not done:

        # Epsilon-greedy action selection
if np.random.random() <= epsilon:

            action = env.action_space.sample()

        else:

            q_values = model.predict(
state.reshape(1, -1),                verbose=0
            )

            action = int(np.argmax(q_values[0]))

        # Take action
        next_state, reward, terminated, truncated, info = env.step(action)

        done = terminated or truncated

        next_state = np.asarray(
next_state,            dtype=np.float32
```

```
        )

        # Store experience
        memory.append(
(                 state,
action,
reward,
next_state,
done
            )
        )

        state = next_state

        total_reward += reward

        # ------------------------------------------------
# TRAIN FROM REPLAY MEMORY
        # ------------------------------------------------

        if len(memory) >= batch_size:

            minibatch = random.sample(
memory,                 batch_size
            )

            states = np.asarray(
                [experience[0] for experience in minibatch],
dtype=np.float32
            )

            actions = np.asarray(
                [experience[1] for experience in minibatch],
dtype=np.int32
            )

            rewards = np.asarray(
                [experience[2] for experience in minibatch],
dtype=np.float32
            )

            next_states = np.asarray(
                [experience[3] for experience in minibatch],
dtype=np.float32
```

```
            )

            dones = np.asarray(
                [experience[4] for experience in minibatch],
dtype=bool
            )

            # Current Q-values
current_q = model.predict(
states,                verbose=0
            )

            # Next Q-values
next_q = target_model.predict(
next_states,                verbose=0
            )

            # Update target Q-values
for i in range(batch_size):

                if dones[i]:

                    target = rewards[i]

                else:

                    target = (
rewards[i]
                        + gamma * np.max(next_q[i])
                    )

                current_q[i, actions[i]] = target

            # Train neural network
model.fit(                states,
current_q,
epochs=1,
verbose=0
            )

    # ------------------------------------------------
# REDUCE EXPLORATION
    # ------------------------------------------------
```

```
    if epsilon > epsilon_min:

        epsilon *= epsilon_decay

    epsilon = max(
epsilon,
epsilon_min
    )

    # --------------------------------------------------
# UPDATE TARGET NETWORK
    # --------------------------------------------------

    if (episode + 1) % 10 == 0:

        target_model.set_weights(
model.get_weights()
        )

    # Store reward
    rewards_history.append(total_reward)

    print(         f"Episode {episode +
1:3d} | "         f"Reward:
{total_reward:3.0f} | "
f"Epsilon: {epsilon:.3f}"
    )


# Close environment
env.close()

# --------------------------------------------------
# 6. PERFORMANCE GRAPH
# --------------------------------------------------

plt.figure(figsize=(10, 5))

plt.plot(
rewards_history,
label="DQN Reward"
)
```

```
plt.xlabel("Episode") plt.ylabel("Total
Reward")

plt.title(
    "Deep Q-Network (DQN) Performance on CartPole"
)

plt.legend()

plt.grid(True, linestyle="--", alpha=0.6)

plt.tight_layout()

plt.show()

# ------------------------------------------------
# 7. FINAL RESULT
# ------------------------------------------------

print("\n======================================") print("DQN
TRAINING COMPLETED SUCCESSFULLY!")
print("======================================")

print("Total Episodes :", episodes) print("Best
Reward    :", max(rewards_history))
print("Average Reward :", round(np.mean(rewards_history), 2))
```

OUTPUT:

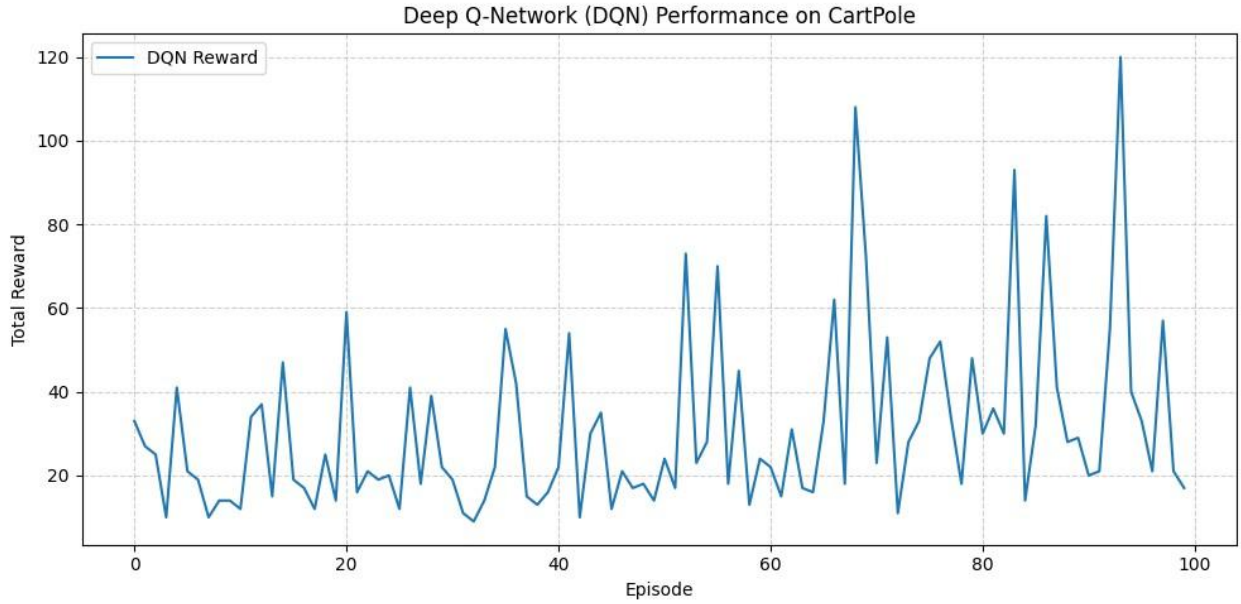
```
State Size : 4
Action Size: 2

DQN model created successfully!
Episode   1 | Reward:  33 | Epsilon: 0.995
Episode   2 | Reward:  27 | Epsilon: 0.990
Episode   3 | Reward:  25 | Epsilon: 0.985
Episode   4 | Reward:  10 | Epsilon: 0.980
Episode   5 | Reward:  41 | Epsilon: 0.975
Episode   6 | Reward:  21 | Epsilon: 0.970
Episode   7 | Reward:  19 | Epsilon: 0.966
```

```
Episode   8 | Reward:  10 | Epsilon: 0.961
Episode   9 | Reward:  14 | Epsilon: 0.956
Episode  10 | Reward:  14 | Epsilon: 0.951
Episode  11 | Reward:  12 | Epsilon: 0.946
Episode  12 | Reward:  34 | Epsilon: 0.942
Episode  13 | Reward:  37 | Epsilon: 0.937
Episode  14 | Reward:  15 | Epsilon: 0.932
Episode  15 | Reward:  47 | Epsilon: 0.928
Episode  16 | Reward:  19 | Epsilon: 0.923
Episode  17 | Reward:  17 | Epsilon: 0.918
Episode  18 | Reward:  12 | Epsilon: 0.914
Episode  19 | Reward:  25 | Epsilon: 0.909
Episode  20 | Reward:  14 | Epsilon: 0.905
Episode  21 | Reward:  59 | Epsilon: 0.900
Episode  22 | Reward:  16 | Epsilon: 0.896
Episode  23 | Reward:  21 | Epsilon: 0.891
Episode  24 | Reward:  19 | Epsilon: 0.887
Episode  25 | Reward:  20 | Epsilon: 0.882
Episode  26 | Reward:  12 | Epsilon: 0.878
Episode  27 | Reward:  41 | Epsilon: 0.873
Episode  28 | Reward:  18 | Epsilon: 0.869
Episode  29 | Reward:  39 | Epsilon: 0.865
Episode  30 | Reward:  22 | Epsilon: 0.860
Episode  31 | Reward:  19 | Epsilon: 0.856
Episode  32 | Reward:  11 | Epsilon: 0.852
Episode  33 | Reward:   9 | Epsilon: 0.848
Episode  34 | Reward:  14 | Epsilon: 0.843
Episode  35 | Reward:  22 | Epsilon: 0.839
Episode  36 | Reward:  55 | Epsilon: 0.835
Episode  37 | Reward:  42 | Epsilon: 0.831
Episode  38 | Reward:  15 | Epsilon: 0.827
Episode  39 | Reward:  13 | Epsilon: 0.822
Episode  40 | Reward:  16 | Epsilon: 0.818
Episode  41 | Reward:  22 | Epsilon: 0.814
Episode  42 | Reward:  54 | Epsilon: 0.810
Episode  43 | Reward:  10 | Epsilon: 0.806
Episode  44 | Reward:  30 | Epsilon: 0.802
Episode  45 | Reward:  35 | Epsilon: 0.798
```

```
Episode  46 | Reward:  12 | Epsilon: 0.794
Episode  47 | Reward:  21 | Epsilon: 0.790
Episode  48 | Reward:  17 | Epsilon: 0.786
Episode  49 | Reward:  18 | Epsilon: 0.782
Episode  50 | Reward:  14 | Epsilon: 0.778
Episode  51 | Reward:  24 | Epsilon: 0.774
Episode  52 | Reward:  17 | Epsilon: 0.771
Episode  53 | Reward:  73 | Epsilon: 0.767
Episode  54 | Reward:  23 | Epsilon: 0.763
Episode  55 | Reward:  28 | Epsilon: 0.759
Episode  56 | Reward:  70 | Epsilon: 0.755
Episode  57 | Reward:  18 | Epsilon: 0.751
Episode  58 | Reward:  45 | Epsilon: 0.748
Episode  59 | Reward:  13 | Epsilon: 0.744
Episode  60 | Reward:  24 | Epsilon: 0.740
Episode  61 | Reward:  22 | Epsilon: 0.737
Episode  62 | Reward:  15 | Epsilon: 0.733
Episode  63 | Reward:  31 | Epsilon: 0.729
Episode  64 | Reward:  17 | Epsilon: 0.726
Episode  65 | Reward:  16 | Epsilon: 0.722
Episode  66 | Reward:  33 | Epsilon: 0.718
Episode  67 | Reward:  62 | Epsilon: 0.715
Episode  68 | Reward:  18 | Epsilon: 0.711
Episode  69 | Reward: 108 | Epsilon: 0.708
Episode  70 | Reward:  72 | Epsilon: 0.704
Episode  71 | Reward:  23 | Epsilon: 0.701
Episode  72 | Reward:  53 | Epsilon: 0.697
Episode  73 | Reward:  11 | Epsilon: 0.694
Episode  74 | Reward:  28 | Epsilon: 0.690
Episode  75 | Reward:  33 | Epsilon: 0.687
Episode  76 | Reward:  48 | Epsilon: 0.683
Episode  77 | Reward:  52 | Epsilon: 0.680
Episode  78 | Reward:  34 | Epsilon: 0.676
Episode  79 | Reward:  18 | Epsilon: 0.673
Episode  80 | Reward:  48 | Epsilon: 0.670
Episode  81 | Reward:  30 | Epsilon: 0.666
Episode  82 | Reward:  36 | Epsilon: 0.663
Episode  83 | Reward:  30 | Epsilon: 0.660
```

```
Episode  84 | Reward:  93 | Epsilon: 0.656
Episode  85 | Reward:  14 | Epsilon: 0.653
Episode  86 | Reward:  32 | Epsilon: 0.650
Episode  87 | Reward:  82 | Epsilon: 0.647
Episode  88 | Reward:  41 | Epsilon: 0.643
Episode  89 | Reward:  28 | Epsilon: 0.640
Episode  90 | Reward:  29 | Epsilon: 0.637
Episode  91 | Reward:  20 | Epsilon: 0.634
Episode  92 | Reward:  21 | Epsilon: 0.631
Episode  93 | Reward:  55 | Epsilon: 0.627
Episode  94 | Reward: 120 | Epsilon: 0.624
Episode  95 | Reward:  40 | Epsilon: 0.621
Episode  96 | Reward:  33 | Epsilon: 0.618
Episode  97 | Reward:  21 | Epsilon: 0.615
Episode  98 | Reward:  57 | Epsilon: 0.612
Episode  99 | Reward:  21 | Epsilon: 0.609
Episode 100 | Reward:  17 | Epsilon: 0.606
```




```
======================================
DQN TRAINING COMPLETED SUCCESSFULLY!
======================================
Total Episodes : 100
Best Reward    : 120.0
Average Reward : 30.59
```

RESULT

The Deep Q-Network was successfully implemented using TensorFlow and Keras, and the agent was trained to learn suitable actions in the Reinforcement Learning environment.